



中山大學

SUN YAT-SEN UNIVERSITY

第二章 数字系统、运算和编码

于帅

计算机学院

2026 年 3 月

主要内容

1 数的进制和转换

2 有符号数及其算术运算

3 BCD和数字编码

4 校验码

主要内容

1 数的进制和转换

2 有符号数及其算术运算

3 BCD和数字编码

4 校验码



数的进制和转换

- 思考：5132.13是什么意思？

- 取决于基（按位计数制）

- 如果基为10（十进制）

$$5132.13_{10} = 5 \times 10^3 + 1 \times 10^2 + 3 \times 10^1 + 2 \times 10^0 + 1 \times 10^{-1} + 3 \times 10^{-2}$$

- 如果基为6（六进制）

$$5132.13_6 = 5 \times 6^3 + 1 \times 6^2 + 3 \times 6^1 + 2 \times 6^0 + 1 \times 6^{-1} + 3 \times 6^{-2}$$

我们经常使用
下标来表示基



数的进制和转换

- 任意 N 进制数展开式的普遍形式：

$$D = \sum k_i N^i$$

其中： k_i 是第 i 位的系数；

k_i 可以是 $0 \sim N-1$ 中的任何一个；

N 称为计数的基；

N^i 称为第 i 位的权。



数的进制和转换

- 十进制数 (Decimal Numbers)

- 十进制数中每个数字的“位置”表示所代表数量的大小，并可分配权

例：5236.71₁₀

十进制：基为10!

权是基的整数
(位置) 次幂!

$$= 5 \times 1000 + 2 \times 100 + 3 \times 10 + 6 \times 1 + 7 \times 0.1 + 1 \times 0.01$$

$$= 5 \times 10^3 + 2 \times 10^2 + 3 \times 10^1 + 6 \times 10^0 + 7 \times 10^{-1} + 1 \times 10^{-2}$$



数的进制和转换

• 十进制数 (Decimal Numbers)

➤ 对任意十进制数，基为10：

$$\begin{aligned} N &= a_{n-1}a_{n-2} \cdots a_1a_0.a_{-1}a_{-2} \cdots a_{-m} \\ &= a_{n-1} \times 10^{n-1} + a_{n-2} \times 10^{n-2} + \cdots + a_1 \times 10 + a_0 \times 10^0 + a_{-1} \times 10^{-1} \\ &\quad + a_{-2} \times 10^{-2} + \cdots + a_{-m} \times 10^{-m} \\ &= \sum a_i \times 10^i \end{aligned}$$

注意：首位为第0位！



数的进制和转换

• 二进制数 (Binary Numbers)

➤ 带有0、1两种数字的二进制系统是基为2的系统

DECIMAL NUMBER	BINARY NUMBER			
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1
10	1	0	1	0
11	1	0	1	1
12	1	1	0	0
13	1	1	0	1
14	1	1	1	0
15	1	1	1	1

$$\begin{aligned} N &= a_{n-1}a_{n-2} \cdots a_1a_0.a_{-1}a_{-2} \cdots a_{-m} \\ &= a_{n-1} \times 2^{n-1} + a_{n-2} \times 2^{n-2} + \cdots + a_1 \times 2^1 + a_0 \times 2^0 \\ &\quad + a_{-1} \times 2^{-1} + a_{-2} \times 2^{-2} + \cdots + a_{-m} \times 2^{-m} \\ &= \sum a_i \times 2^i \end{aligned}$$



数的进制和转换

• 十六进制数 (Hexadecimal Numbers)

- 更少的位数表达同样多的数字，广泛应用于计算机存储器、网络等地址标识
- A(10),B(11),C(12),D(13),E(14),F(15);

$$\begin{aligned} N &= a_{n-1}a_{n-2} \cdots a_1a_0.a_{-1}a_{-2} \cdots a_{-m} \\ &= a_{n-1} \times 16^{n-1} + a_{n-2} \times 16^{n-2} + \cdots + a_1 \times 16 + a_0 \times 16^0 + a_{-1} \times 16^{-1} + a_{-2} \times 16^{-2} \\ &\quad + \cdots + a_{-m} \times 16^{-m} \\ &= \sum a_i \times 16^i \end{aligned}$$



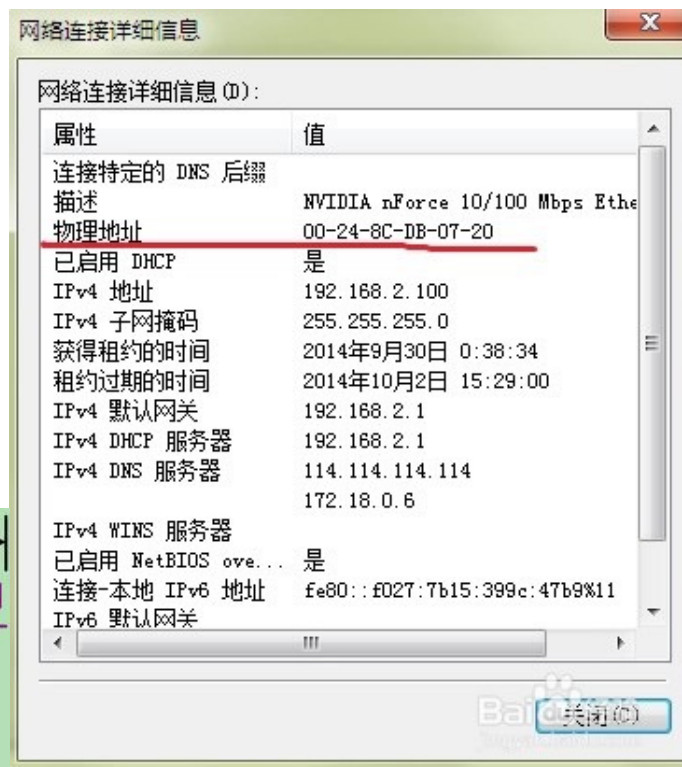
数的进制和转换

• 十六进制数 (Hexadecimal Numbers)

➤ 物理地址 (MAC) :

-12个16进制数

二进制数标识位置过长!





数的进制和转换

• 八进制数 (Octal Numbers)

- 八进制数的每一位有0~7八个不同的数码，计数基为8。低位和相邻的高位之间的进位关系是“逢八进一”；

例：将八进制数12.4展开并转换为十进制数

$$\begin{aligned}(12.4)_8 &= 1 \times 8^1 + 2 \times 8^0 + 4 \times 8^{-1} \\ &= (10.5)_{10}\end{aligned}$$



数的进制和转换

- 进制转换

- 二进制 → 十进制

- 展开相加：二进制数中所有位是1的权相加

例：101011.11₂

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2}$$

$$= 32 + 0 + 8 + 0 + 2 + 1 + \frac{1}{2} + \frac{1}{4}$$

$$= 43.75_{10}$$



数的进制和转换

• 进制转换

➤ 十进制 → 二进制

- 整数部分：除2法

注意：位的位置

$$67_{10} = 1000011_2$$

最上方的数值贴近小数点

$$2 \overline{) 67}$$

.....

$$\text{余数} = 1 = k_0$$

$$2 \overline{) 33}$$

.....

$$\text{余数} = 1 = k_1$$

$$2 \overline{) 16}$$

.....

$$\text{余数} = 0 = k_2$$

$$2 \overline{) 8}$$

.....

$$\text{余数} = 0 = k_3$$

$$2 \overline{) 4}$$

.....

$$\text{余数} = 0 = k_4$$

$$2 \overline{) 2}$$

.....

$$\text{余数} = 0 = k_5$$

$$2 \overline{) 1}$$

.....

$$\text{余数} = 1 = k_6$$

0



数的进制和转换

• 进制转换

➤ 十进制 → 二进制

- 小数部分：乘2法

注意：位的位置

$$0.625_{10} = 0.101_2$$

0.625

× 2

1.250

0.250

× 2

0.500

0.500

× 2

1.000

整数部分=1= k_1

整数部分=0= k_2

整数部分=1= k_3



数的进制和转换

• 练习: $173.8125_{10} = ()_2$?

2	173	1
2	86	0
2	43	1
2	21	1
2	10	0
2	5	1
2	2	0
2	1	1
0		

$(173)_{10} = (10101101)_2$

0.8125	
X 2	
1.6250	$1 = k_{-1}$
0.6250	
X 2	
1.2500	$1 = k_{-2}$
0.2500	
X 2	
0.5000	$0 = k_{-3}$
0.5000	
X 2	
1.0000	$1 = k_{-4}$

$$(0.8125)_{10} = (0.1101)_2$$

$$173.8125_{10} = 10101101.1101_2$$



数的进制和转换

- 进制转换

- 二进制 → 十六进制

- 每四位一组二进制数代替一位十六进制数

例：将二进制数010100011011.10110010转换为十六进制数。

0101, 0001, 1011.1011, 0010

↓ ↓ ↓ ↓ ↓

(5 1 B , B 2)₁₆



数的进制和转换

- 进制转换

- 十六进制 → 二进制

- 一位十六进制数由一组四位二进制数代替

例：将十六进制数8FC.6A转换为二进制数。

(	8		F		C	.	6		A	)	₁₆
	↓		↓		↓		↓		↓		
	1000	,	1111	,	1100	.	0110	,	1010		



数的进制和转换

- 进制转换

- 二进制 → 八进制

- 每三位一组二进制数代替一位八进制数

例：将二进制数010110.101010转换为八进制数

010, 110. 101, 010
↓ ↓ ↓ ↓
(2 6 . 5 2)₈



数的进制和转换

- 进制转换

- 八进制 → 二进制

- 一位八进制数由一组三位二进制数代替

例：将八进制数52.43转换为二进制数。

(5 2 . 4 3)₈

↓ ↓ ↓ ↓

101, 010. 100, 011



数的进制和转换

- **进制转换**

- **十六进制 → 十进制**

- 各位按权展开后相加。

- **十进制 → 十六进制**

- 先转换成二进制数，再转换成等值的十六进制数。

主要内容

1 数的进制和转换

2 有符号数及其算术运算

3 BCD和数字编码

4 校验码



有符号数及其算数运算

- 有符号数（原码形式）

- 二进制数的正、负表示方法通常采用的是在二进制数的前面增加一位符号位：**0表示正数，1表示负数**

00011001

↑
符号位

数值位

10011001

↑
符号位

数值位

注意与二进制数的区别



有符号数及其算数运算

- 有符号数（原码形式）

➤ 例：

十进制数	有符号数
25	0001 1001
-25	1001 1001
39	0010 0111
-39	1010 0111
0	0000 0000 1000 0000

问题1：原码形式 $25 + (-25) = ?$



有符号数及其算数运算

- 有符号数的数值范围

➤ 取决于数的位数 (n)

$$-(2^{n-1})+1 \sim +(2^{n-1}-1) \quad \text{WHY?}$$

例:

$$n = 8: \quad -127 \sim 127$$

$$11111111 \sim 10000000 \quad 01111111 \sim 00000000$$

$$-0 \sim -127$$

$$127 \sim +0$$

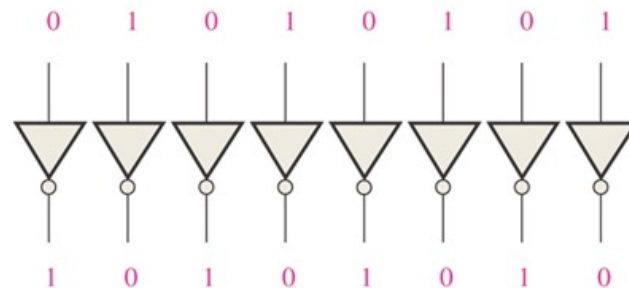
问题2: +0 和 -0 是什么鬼?



有符号数及其算数运算

• 二进制数的反码

- 改变数中的每一位得到**反码**
 - (并行) 反相器: 数字电路求反码



反相器

注意区别二进制数的反码
和有符号数的反码形式

• 有符号数反码形式

- 正数的反码形式: 与原码相同
- 负数的反码形式: 对应正数的反码

二进制数



有符号数及其算数运算

• 练习

十进制数	有符号数	反码形式
25	0001 1001	0001 1001
-25	1001 1001	1110 0110
39	0010 0111	0010 0111
-39	1010 0111	1101 1000
0	0000 0000 1000 0000	0000 0000 1111 1111

问题3：反码形式 $25 + (-25) = ?$



有符号数及其算数运算

- 有符号数反码的数值范围

$$-(2^{n-1})+1 \sim +(2^{n-1}-1) \quad \text{WHY?}$$

Example

$$n = 8: \quad -127 \sim 127$$

$$11111111 \sim 10000000 \quad 01111111 \sim 00000000$$

$$-0 \sim -127$$

$$127 \sim +0$$



有符号数及其算数运算

• 二进制数的补码

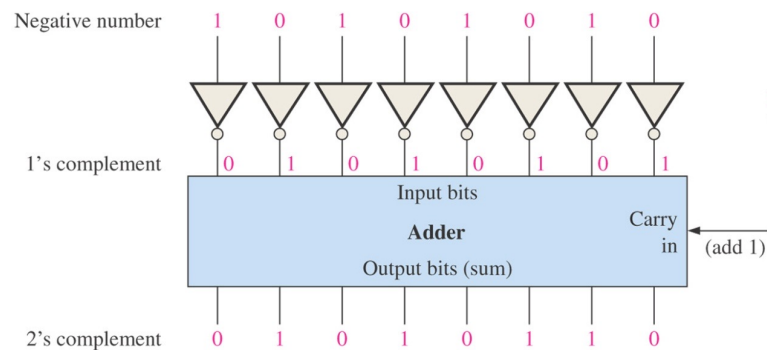
➤ 反码加 1 得到补码

- (并行) 反相器和加法器：数字电路求补码

• 有符号数补码形式

➤ 正数的补码：与原码相同

➤ 负数的补码：对应正数的补码



反相器和加法器得到补码



有符号数及其算数运算

• 练习

十进制数	有符号数 (原码)	反码形式	补码形式
25	0001 1001	0001 1001	0001 1001
-25	1001 1001	1110 0110	1110 0111
39	0010 0111	0010 0111	0010 0111
-39	1010 0111	1101 1000	1101 1001
0	0000 0000 1000 0000	0000 0000 1111 1111	0000 0000



有符号数及其算数运算

- 有符号数补码的数值范围

$$-(2^{n-1}) \sim +(2^{n-1} - 1) \quad \text{WHY?}$$

Example

$$n = 8: \quad -128 \sim 127$$

$$11111111 \sim 10000000 \quad 01111111 \sim 00000000$$

$$-1 \sim -128$$

$$127 \sim 0$$



有符号数及其算数运算

二进制数

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

有符号数 (原码形式)

-7	-6	-5	-4	-3	-2	-1	-0	0	1	2	3	4	5	6	7
1111	1110	1101	1100	1011	1010	1001	1000	0000	0001	0010	0011	0100	0101	0110	0111

反码形式

-7	-6	-5	-4	-3	-2	-1	-0	0	1	2	3	4	5	6	7
1000	1001	1010	1011	1100	1101	1110	1111	0000	0001	0010	0011	0100	0101	0110	0111

补码形式

-8	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7
1000	1001	1010	1011	1100	1101	1110	1111	0000	0001	0010	0011	0100	0101	0110	0111



有符号数及其算数运算

- 为何数字电路中使用补码

WHY?

- 数字电路中使用原码做减法运算

-首先需要比较两数绝对值的大小，然后以绝对值大的一个作为被减数、绝对值小的一个作为减数，求出差值，并以绝对值大的一个数的符号作为差值的符号。

操作繁琐：需要使用数值比较电路和减法运算电路。



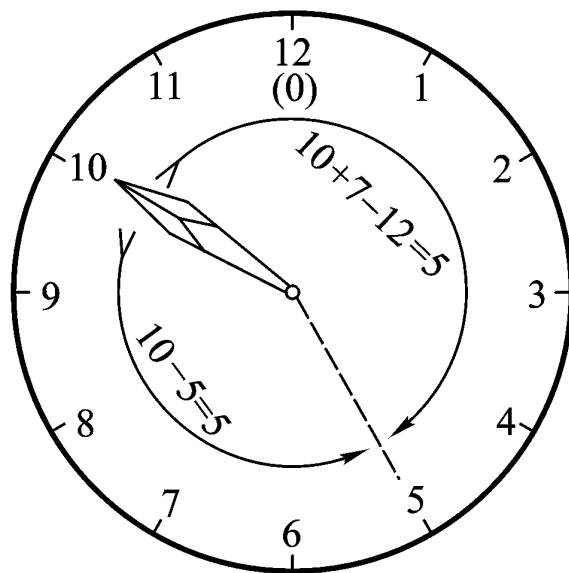
有符号数及其算数运算

• 为何数字电路中使用补码

WHY?

➤ 数字电路中使用“补码相加”做减法运算

-因为5和7相加正好等于产生进位的模数12，所以称7为-5对模12的补数，也称为补码 (complement)。



$$10 - 5 = 5$$

$$(10 + 7) - 12 = 5$$

舍弃进位

10-5的减法运算可以用10+7的加法运算代替。



有符号数及其算数运算

- 为何数字电路中使用补码

WHY?

- 数字电路中使用“补码相加”做减法运算

- 在舍弃进位的条件下，减去某个数可以用加上它的补码来代替。这个结论同样适用于二进制数的运算。



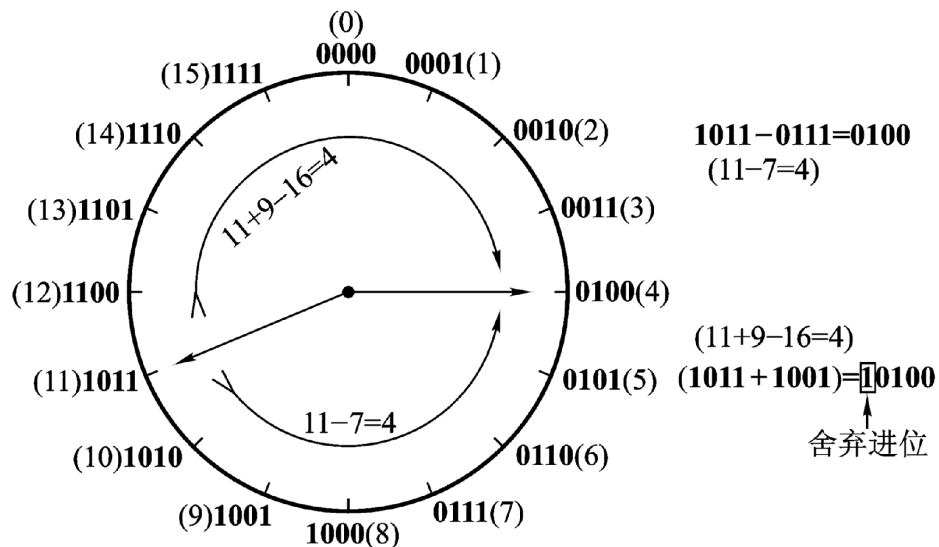
有符号数及其算数运算

• 为何数字电路中使用补码

WHY?

➤ 数字电路中使用“补码相加”做减法运算

-例如：1011-0111=0100的减法运算，在舍弃进位的条件下，可以用1011+1001=0100的加法运算代替。



计算机在所有算数运算中都使用补码来表示负整数

1001是0111对模16的补码



有符号数及其算数运算

- 有符号数的十进制值

- 原码形式 → 十进制值

-符号位确定正负，系数为1加权相加

例：有符号数10100111, 00100111 =?

$$10100111 = (-1)^1 \times (1 \times 2^5 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) = -39$$

$$00100111 = (-1)^0 \times (1 \times 2^5 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0) = 39$$



有符号数及其算数运算

• 有符号数的十进制值

➤ 反码形式 → 十进制值

-正数：所有系数为1的位加权相加

-负数：1.符号位权赋负值；2.所有系数为1（包括符号位）加权相加；

3.再加 1

代表反码形式

$$11011000_{1C} = ?$$

$$00100111_{1C} = ?$$

$$11011000_{1C} = \underline{1 \times (-2^7) + 1 \times 2^6 + 1 \times 2^4 + 1 \times 2^3 + 1} = -39$$

$$00100111_{1C} = 0 \times (-2^7) + 1 \times 2^5 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 39$$



有符号数及其算数运算

- 有符号数的十进制值

- 补码形式 → 十进制值

- 正数：所有系数为1的位加权相加

- 负数：1.符号位权赋负值；2.所有系数为1（包括符号位）加权相加

$$11011001_{2C} = ?$$

$$00100111_{2C} = ?$$

代表补码形式

$$11011001_{2C} = 1 \times (-2^7) + 1 \times 2^6 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^0 = -39$$

$$00100111_{2C} = 0 \times (-2^7) + 1 \times 2^5 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 39$$



有符号数及其算数运算

- 练习:

$$00010111_{1C} = +23$$

$$11101000_{1C} = -23$$

$$01010110_{2C} = +86$$

$$10101010_{2C} = -86$$



有符号数及其算数运算

- 二进制算数运算

- 二进制算术运算和十进制算术运算的规则基本相同；
- 唯一的区别在于二进制数是“逢二进一”而不是十进制数的“逢十进一”；



有符号数及其算数运算

• 二进制算数运算

➤ 例如：两个二进制数1001和0101的算术运算有：

加法运算

$$\begin{array}{r} 1001 \\ + 0101 \\ \hline 1110 \end{array}$$

减法运算

$$\begin{array}{r} 1001 \\ - 0101 \\ \hline 0100 \end{array}$$

乘法运算

$$\begin{array}{r} 1001 \\ \times 0101 \\ \hline 1001 \\ 0000 \\ 1001 \\ 0000 \\ \hline 0101101 \end{array}$$

除法运算

$$\begin{array}{r} 1.11\dots \\ 0101 \overline{) 1001} \\ \underline{0101} \\ 1000 \\ \underline{0101} \\ 0110 \\ \underline{0101} \\ 0010 \end{array}$$



有符号数及其算数运算

• 有符号数的算数运算

计算机在所有算数运算中都使用补码来表示负整数

➤ 加法：将两个数加起来，并舍去最后的进位

$$\begin{array}{r} 0000100 \\ 00000111 \quad (+7_{10}) \\ +00000100 \quad (+4_{10}) \\ \hline 00001011 \quad (+11_{10}) \end{array}$$

两个正数相加

$$\begin{array}{r} 1111111 \\ 11111011 \quad (-5_{10}) \\ +11110111 \quad (-9_{10}) \\ \hline 111110010 \quad (-14_{10}) \end{array}$$

两个负数相加

舍去进位

$$\begin{array}{r} 1111110 \\ 00001111 \quad (+15_{10}) \\ +11111010 \quad (-6_{10}) \\ \hline 100001001 \quad (+9_{10}) \end{array}$$

正负相加，正数绝对值大

$$\begin{array}{r} 0000000 \\ 00010000 \quad (+16_{10}) \\ +11101000 \quad (-24_{10}) \\ \hline 11111000 \quad (-8_{10}) \end{array}$$

正负相加，负数绝对值大。

舍去进位



有符号数及其算数运算

• 有符号数的算数运算

注意此处为减数（二进制数）的补码

➤ 减法：取减数的补码相加，并舍去最后的进位

例：00001000 (+8₁₀) - 00000011 (+3₁₀) = ?

00001100 (+12₁₀) - 11110111 (-9₁₀) = ?

1 1 1 1 0 0 0
00001000
+11111101
100000101

舍去进位

0 0 0 1 0 0 0
00001100
+00001001
00010101

相当于十进制数
-3的补码形式



有符号数及其算数运算

• 有符号数的算数运算

➤ 溢出条件：和的位数超出

	1 1 1 1 0 0 0	
	01111101	(+125 ₁₀)
	+00111010	(+58 ₁₀)
	<u>10110111</u>	(+183 ₁₀)
符号位错误		
		大小不正确

在两个同符号数相加时，它们的绝对值之和不可超过有效数字位所能表示的最大值，否则会得出错误的计算结果。



有符号数及其算数运算

• 练习

➤ 用二进制补码运算求 $13+10$ 、 $13-10$ 、 $-13+10$ 、 $-13-10$ 。

首先确定位数！

$$\begin{array}{r} +13 \quad 0 \ 01101 \\ +10 \quad 0 \ 01010 \\ \hline +23 \quad 0 \ 10111 \end{array}$$

$$\begin{array}{r} -13 \quad 1 \ 10011 \\ +10 \quad 0 \ 01010 \\ \hline -3 \quad 1 \ 11101 \end{array}$$

$$\begin{array}{r} +13 \quad 0 \ 01101 \\ -10 \quad 1 \ 10110 \\ \hline +3 \quad (1) \ 0 \ 00011 \end{array}$$

$$\begin{array}{r} -13 \quad 1 \ 10011 \\ -10 \quad 1 \ 10110 \\ \hline -23 \quad (1) \ 1 \ 01001 \end{array}$$



有符号数及其算数运算

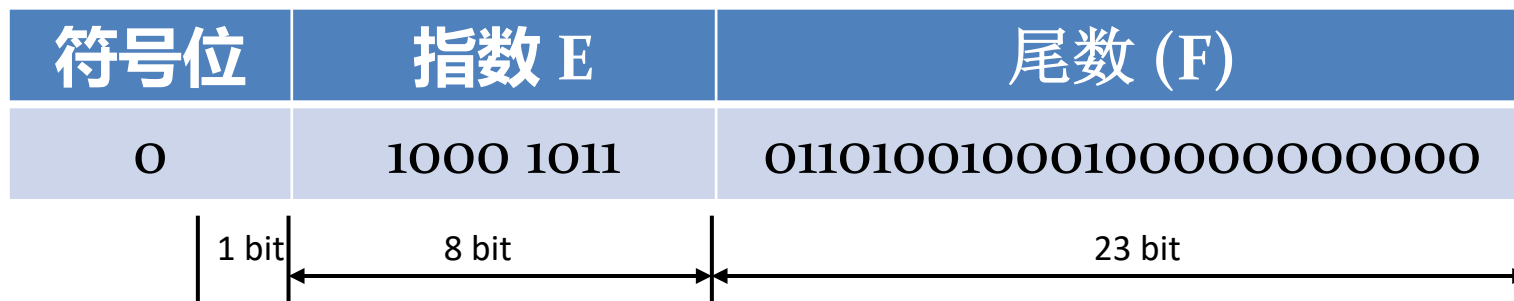
• 浮点数（实数）

➤ 浮点数由两个部分加上一个符号组成。

-尾数 (mantissa) : 浮点数中表示数值大小的部分。

-指数 (exponent) : 表示小数点（或二进制小数点）要移动的位数。

➤ 单精度：32位；双精度：64位；扩展精度：80位





有符号数及其算数运算

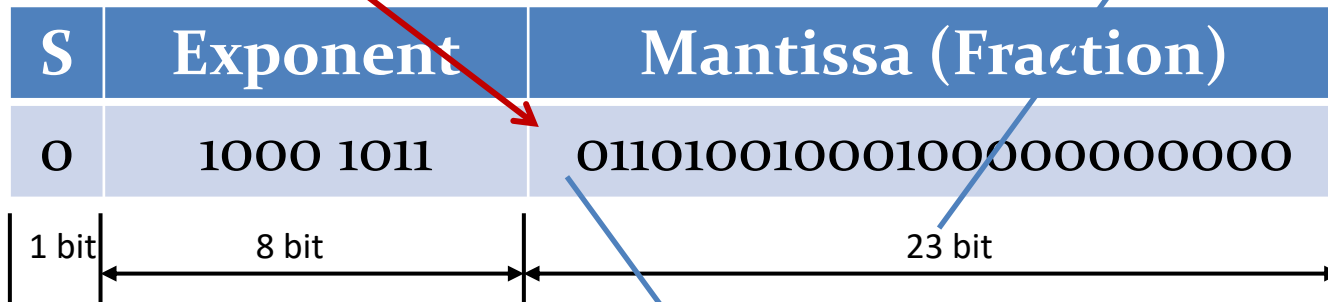
• 单精度浮点数

$$1\ 0110\ 1001\ 0001 = 1.0110\ 1001\ 0001 \times 2^{12} \quad (\text{正数})$$

偏移指数! (为了允许很大很小数)

➤ 指数: $12 + 127 = 139$ -----> 1000 1011。

➤ 尾数: (1) 0110 1001 0001 尾数具有24位! 最左边默认为1 (无占位)



二进制小数点位于23位左边



有符号数及其算数运算

• 练习

➤ 确定以下浮点二进制数的二进制值。

S	Exponent	Mantissa (Fraction)
1	10010001	100011100010000000000000

$$\begin{aligned}\text{Number} &= (-1)^S (1 + F) (2^{E-127}) \\ &= (-1)^1 (1.10001110001) (2^{145-127}) \\ &= -11000111000100000000\end{aligned}$$



有符号数及其算数运算

- 特例

+/- 0

S	Exponent	Mantissa (Fraction)
0/1	00000000	000000000000000000000000

+/- inf

S	Exponent	Mantissa (Fraction)
0/1	11111111 1	000000000000000000000000

NaN

Not a Number

S	Exponent	Mantissa (Fraction)
0/1	11111111 1	non zero

主要内容

1 数的进制和转换

2 有符号数及其算术运算

3 BCD和数字编码

4 校验码



BCD和数字编码

- **二进制码 (Binary Coded Decimal, BCD)**
 - 为了用四位二进制代码表示十进制数的0~9这十个状态,
 - 4位二进制代码一共有十六个 (0000~1111) ,
 - 取其中哪十个以及如何与0~9相对应, 有许多种方案。
- **BCD码分类**
 - 有权码: 8421、2421、5211
 - 无权码: 格雷码、余3码、余3循环码



BCD和数字编码

十进制数 \ 编码种类	8421码 BCD代码	余3码	2421码	5211码	余3循环码
0	0000	0011	0000	0000	0010
1	0001	0100	0001	0001	0110
2	0010	0101	0010	0100	0111
3	0011	0110	0011	0101	0101
4	0100	0111	0100	0111	0100
5	0101	1000	1011	1000	1100
6	0110	1001	1100	1001	1101
7	0111	1010	1101	1100	1111
8	1000	1011	1110	1101	1110
9	1001	1100	1111	1111	1010
权	8421		2421	5211	



BCD和数字编码

• 8421码

- 是BCD码中最常用的一种，提及BCD码总是指8421
- 8421码中每一位的权是固定不变的，它属于**有权代码**

例：将 2496_{10} 转换为BCD码

Decimal Digit	BCD
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1

2 4 9 6
↓ ↓ ↓ ↓
0 0 1 0 0 1 0 0 1 0 0 1 0 1 1 0

注意与转换为二进制数的区别:

1 0 0 1 1 1 0 0 0 0 0 0₂



BCD和数字编码

• 数字编码

➤ 格雷 (Gray) 码

- 无权码，不是算数编码

- 从一个码字到下一个码字仅一位变化

该特征减少了
了出错概率

编码顺序	二进制代码	格雷码
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000



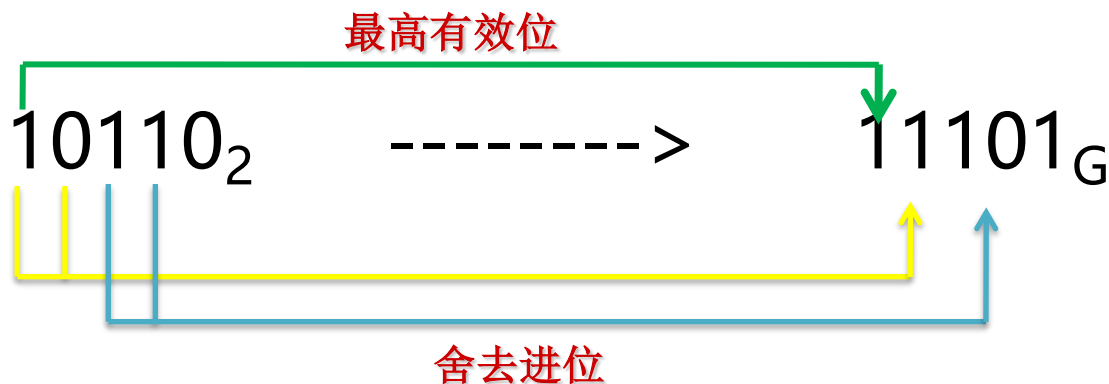
BCD和数字编码

• 数字编码

➤ 二进制数转换为格雷码

-格雷码最高有效位（最左边）等于二进制数相应的最高有效位

-从左到右，加上每一对相邻的二进制编码位，得到下一个格雷码位，并舍去进位





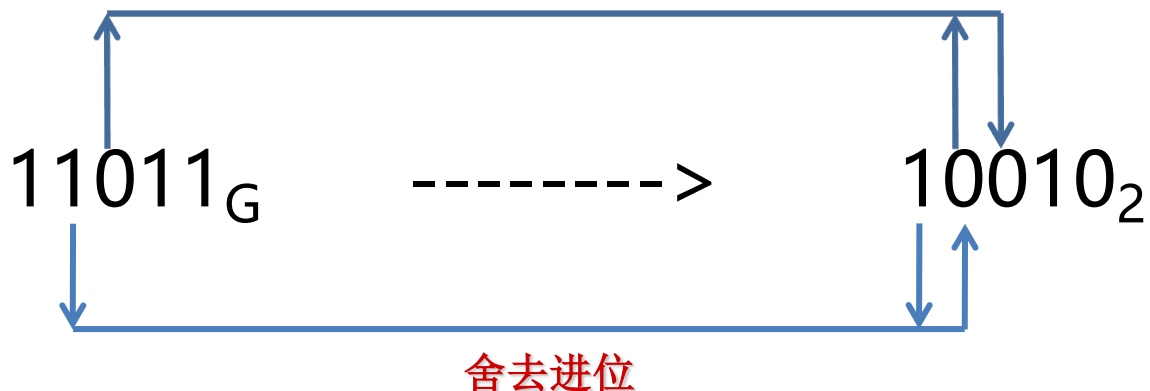
BCD和数字编码

• 数字编码

➤ 格雷码转换为二进制数

-二进制数最高有效位（最左边）等于格雷码中相应的位

-将产生的每个二进制编码位，加上下一个位置的格雷码位，**并舍去进位**





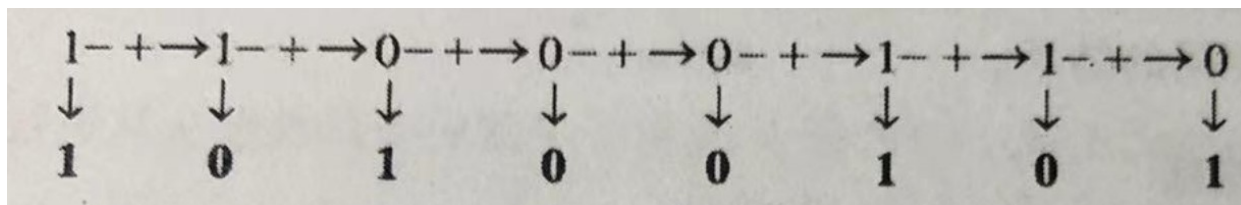
有符号数及其算数运算

- 例题:

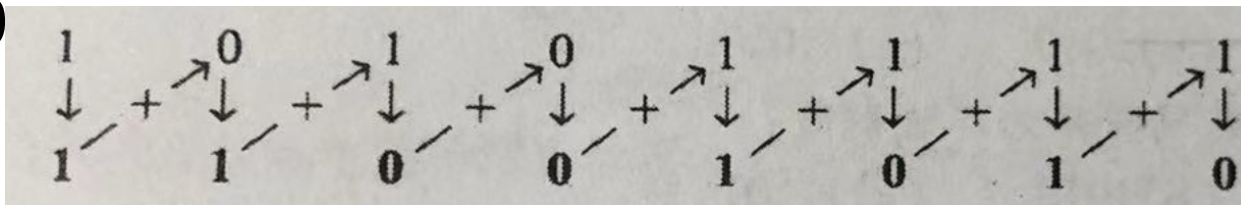
1. 将二进制数 11000110 转换为格雷码

2. 将格雷码10101111转换为二进制数

1. 10100101



2. 11001010





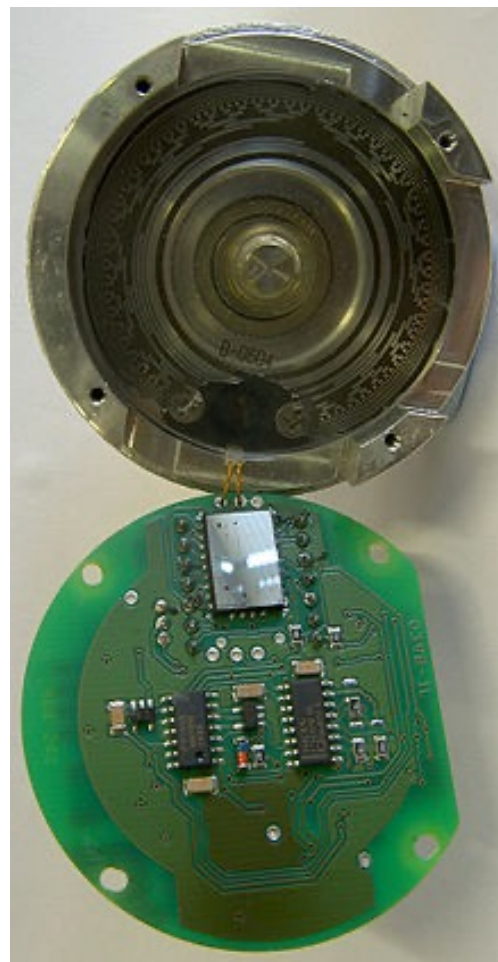
BCD和数字编码

- 数字编码

- 格雷码典型应用



Rotary encoder (旋转编码器)

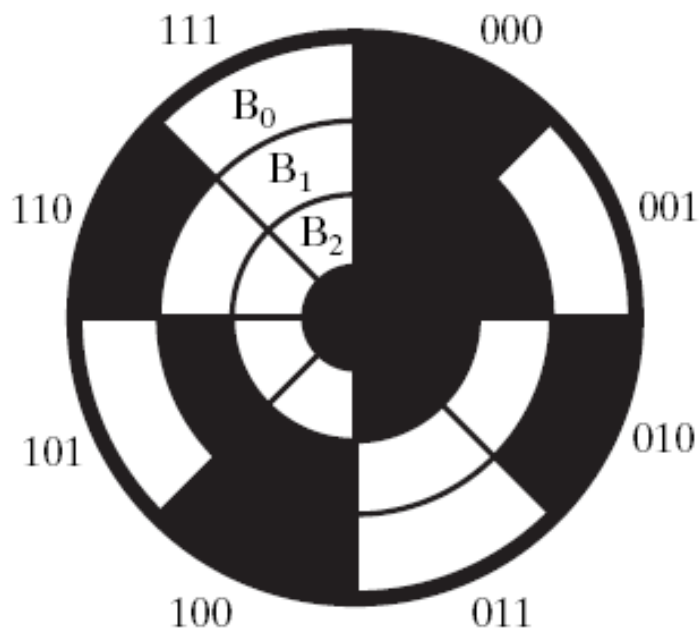




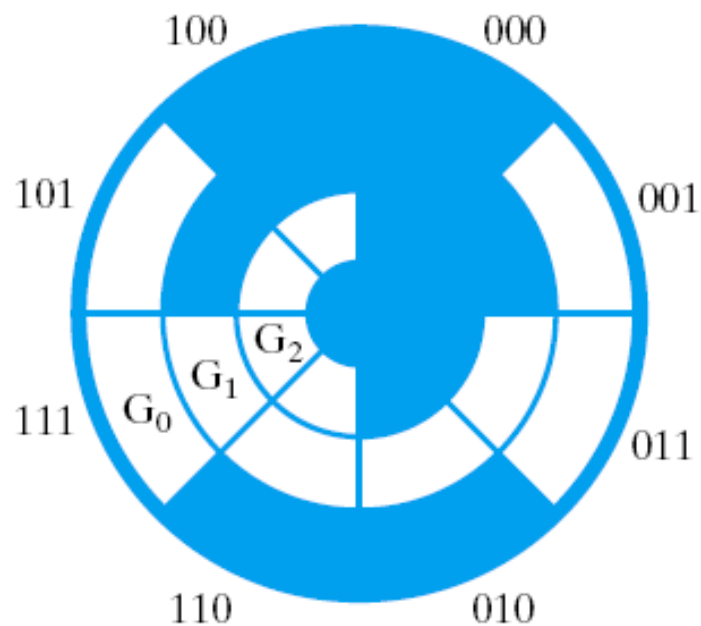
BCD和数字编码

• 数字编码

➤ 格雷码典型应用



(a) Binary Code for Positions 0 through 7



(b) Gray Code for Positions 0 through 7



BCD和数字编码

• 数字编码

➤ 格雷码典型应用

-旋转编码器：在电梯运行过程中，通过旋转编码器检测、软件实时计算以下信号——电梯所在层楼位置、换速点位置、平层点位置，从而进行楼层计数、发出换速信号和平层信号。





BCD和数字编码

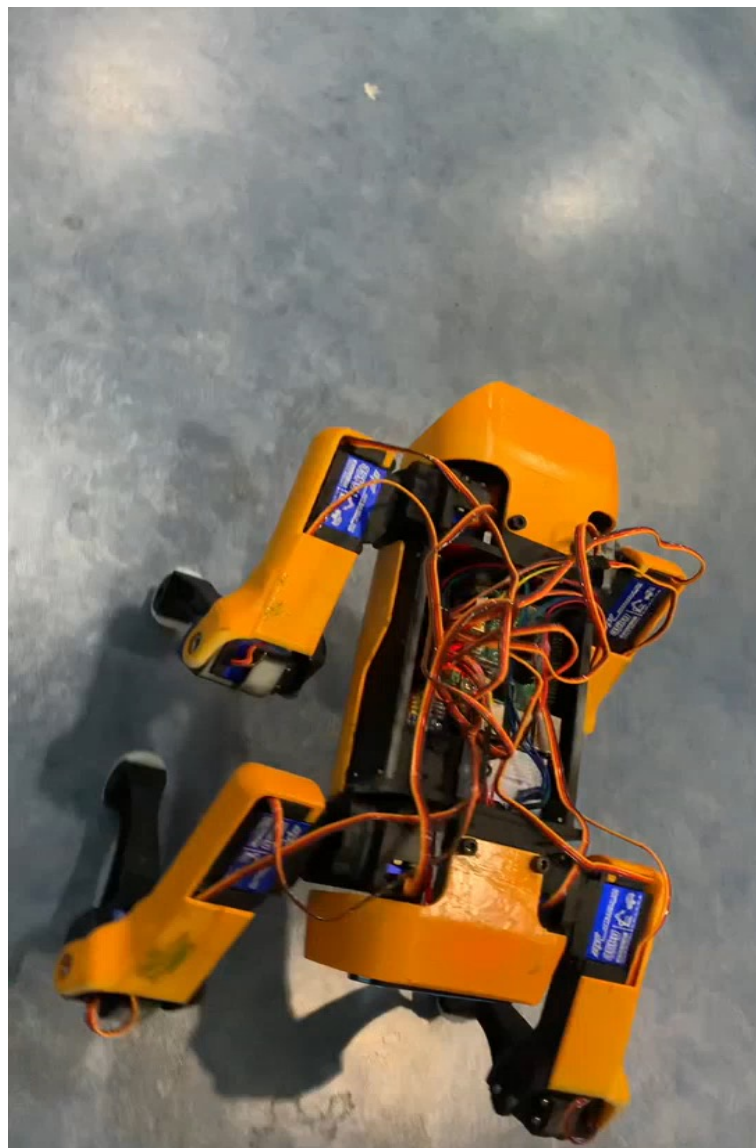
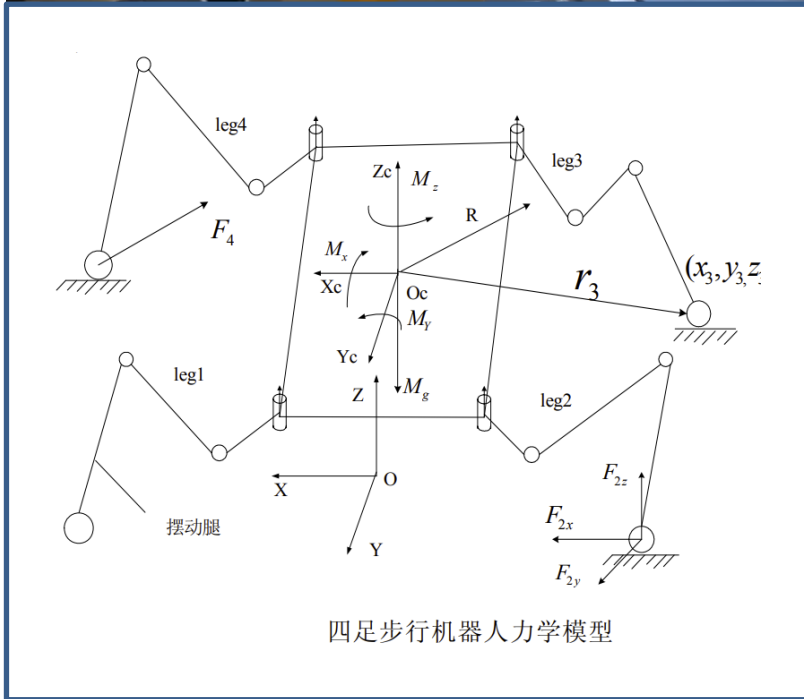
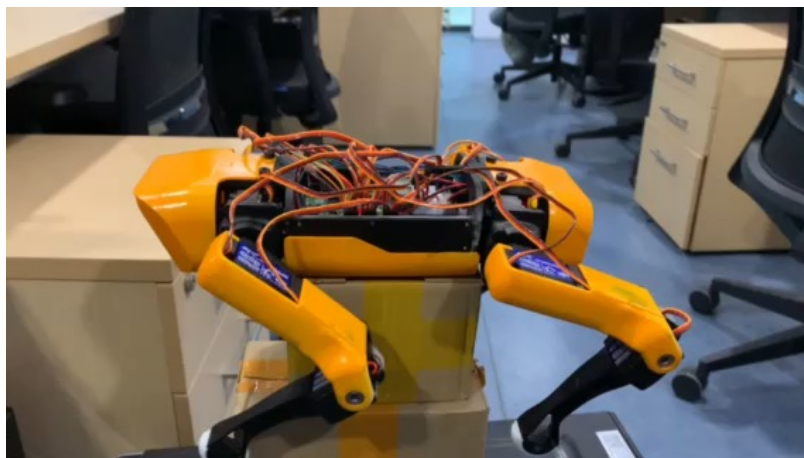


步进电机





BCD和数字编码



主要内容

1 数的进制和转换

2 有符号数及其算术运算

3 BCD和数字编码

4 校验码



校验码

• 错误检测的奇偶校验法

- 奇偶校验位：给出一个数中1的个数是奇数还是偶数；
- 偶校验位：1总个数为偶数；
- 奇校验位：1总个数为奇数；

十进制数	信息码	奇校验码	偶校验码
0	0000	10000	00000
1	0001	00001	10001
2	0010	00010	10010
3	0011	10011	00011
4	0100	00100	10100
5	0101	10101	00101
6	0110	10110	00110
7	0111	00111	10111
8	1000	01000	11000
9	1001	11001	01001

注意：计算1的个数时要
计入校验位



校验码

• 错误检测的奇偶校验法

- 奇偶校验位：给出一个数中1的个数是奇数还是偶数；
- 偶校验位：1总个数为偶数；
- 奇校验位：1总个数为奇数；

例：给下列数加入**偶校验位**：1010, 111000, 101101

1010
↓
01010

111000
↓
1111000

101101
↓
0101101



有符号数及其算数运算

- **练习:**

给下面数组加上偶校验位

1. 1010

2. 111000

3. 101101

4. 1000111001001

5. 101101011111

1. 01010

2. 1111000

3. 0101101

4. 01000111001001

5. 1101101011111



校验码

- **循环冗余校验码 (CRC)**

- 应用于数据通信领域和磁介质存储系统中
- **两部分组成**：数据码（发端数据），生成码（收发端）；
- 数据码包含**数据位和检测位**
- **模2（异或）运算**



校验码

• 循环冗余校验码 (CRC)

➤ 模2 (异或) 运算

普通二进制除法

$$\begin{array}{r} 1111 \\ 110 \overline{) 1011010} \\ \underline{110} \\ 1010 \\ \underline{110} \\ 1001 \\ \underline{110} \\ 110 \\ \underline{110} \\ 0 \end{array}$$

模2二进制除法

$$\begin{array}{r} 11011 \\ 110 \overline{) 1011010} \\ \underline{110} \\ 0111 \\ \underline{110} \\ 00101 \\ \underline{110} \\ 0110 \\ \underline{110} \\ 0 \end{array}$$



校验码

• 循环冗余校验码 (CRC)

➤ 处理步骤:

- 1. 发送端拟发送 m 位数据位给接收端
- 2. 收发端存储相同 n 位生成码 ($n \leq m$)
- 3. 发端将数据位后补齐 n 个0 (生成位)
- 4. 数据位和生成位模-2除校验码
- 5. 余数为0, 直接发送数据位和生成位 (数据码)
- 6. 余数不为0, 令生成位=余数位发送
- 7. 接收端: 接收数据码模-2除校验码
- 8. 余数为0, 没有检测到错误; 余数不为0, 检测到错误, 重传



校验码

• 循环冗余校验码 (CRC)

➤ 例：为数据位 11010011 和生成码 1010 确定CRC。

<u>110100110000</u>	<u>110100110000</u>	<u>110100110100</u>
1010	1010	CRC
	1110	
	1010	
	1000	
	1010	
	1011	
	1010	
	1000	
	1010	
		100



➤ 上例中接收端收到的CRC左边第二位发生错误:

用相同的生成码 (1010) 对接收到的数据码进行错误检测



校验码

• 练习

$$\begin{array}{r} 100100110000 \\ 1010 \end{array}$$

$$\begin{array}{r} 100100110100 \\ 1010 \\ 1100 \\ 1010 \\ 01101 \\ 1010 \\ 1111 \\ 1010 \\ 1010 \\ 1010 \\ 0100 \end{array}$$



课程回顾

- 二、八、十、十六进制及转换
- 二进制数和有符号数
- 原码、反码、**补码**
- 有符号数的运算
- BCD和数字编码
- 校验码



计算机学院
于帅





作业

- Page 54~57

- 11
- 15
- 22
- 23
- 31
- 51
- 56

Page 59~52

- 11
- 15
- 22
- 23
- 31
- 51
- 56